PROJECT LEE — OPERATIONAL INTELLIGENCE, PROJECT OPERATOR, DATABASE, AND DESKTOP COMPLETION PASS

Use the current Project LEE repository and implementation as the source of truth.

Before changing anything

1. Inspect the current implementation thoroughly.
2. Determine what already exists, what is partially implemented, what is duplicated, what is stale, and what is genuinely missing.
3. Create an ordered implementation task plan covering the work below.
4. Then execute those tasks sequentially.
5. Reuse and extend existing architecture rather than creating competing systems.
6. Preserve all existing working functionality, data, Event Log history, Brain state, provider connections, Gmail, Google Drive, Google Calendar, GitHub, Replit project bridges, Android, desktop functionality, backups, CIL, CerbaSeal, system contracts, and existing tests.
7. Do not claim completion because a screen, stub, interface, or documentation entry exists. The actual behavior must work and be tested.

The product principle is:

SIMPLE ON THE OUTSIDE. A MONSTER UNDER THE HOOD.

LEE already has enough major subsystems. The next phase should make those systems work together so effectively that LEE becomes a genuinely useful founder operating environment and technical operator.

The target is a system that knows what is happening, understands why it matters, connects information across systems, tracks what changed, understands commitments and waiting loops, understands project health, can inspect and work on registered projects, prepares safe actions, executes only within authority, surfaces only what deserves attention, starts automatically as a desktop application, and preserves its Brain through updates, crashes, outages, or machine replacement.

PHASE 1 — CORRECT CIL REASONING AUTHORITY

The current Project LEE description still contains outdated behavior describing CIL as an optional reasoning accelerator and permitting LEE to independently fall back to frontier models.

Correct the implementation and documentation.

The canonical normal reasoning path must be:

Owner request
→ Identity
→ Constitution / Policy
→ Intent
→ Query Engine
→ Context Economy
→ CIL API
→ T1 reuse, T2 reuse, or T3 model-route selection
→ LEE Model Router
→ selected execution provider / Replit AI Bridge
→ selected model

→ result
→ LEE

Requirements

- CIL is the mandatory pre-model authority for normal LEE-originated model requests.
- CIL owns T1, T2, and T3 resolution.
- CIL owns model, provider, and route selection for T3.
- LEE's Model Router executes the route selected by CIL.
- The Model Router must not independently choose another model.
- Replit AI Bridge must not silently substitute another model.
- If execution of CIL's selected route fails, return the execution failure to CIL.
- CIL decides whether and how to reroute.
- LEE must not invent a replacement route locally.
- CIL model inventory is read-only visibility and diagnostics only.
- Reading model inventory does not transfer model-selection authority to LEE.
- If CIL is unavailable, local, no-model, context-only, retrieval, database, project, and other non-model capabilities may remain available.
- LEE must not silently bypass unavailable CIL by independently calling a paid or frontier model.

Update System Economics so LEE can measure:

- T1 resolutions
- T2 resolutions
- T3 resolutions
- Frontier Dependency %
- avoided model calls
- actual external model calls
- provider usage
- model usage
- input/output tokens
- latency
- valid estimated spend when current dated pricing evidence exists
- CIL-attributable avoided spend or savings

PHASE 2 — FORMALIZE THE SERVICE PLANE AND MANAGEMENT PLANE

LEE now has the ability to work across projects using Replit/MCP.

Make the distinction between using a system and working on a system explicit in architecture, code, capabilities, tests, and documentation.

Runtime / Service Plane

This is used when LEE consumes another system normally.

Examples:

- LEE → CIL API
- LEE → CerbaSeal API
- LEE → Gmail API
- LEE → future QuantraCore API
- LEE → future Greyline API
- LEE → other Universal Systems APIs

Management / Control Plane

This is used when LEE works on the implementation of a project.

Examples:

- LEE → Replit/MCP Project Bridge → inspect project
- LEE → Project Bridge → read files
- LEE → Project Bridge → inspect logs
- LEE → Project Bridge → run tests
- LEE → Project Bridge → preview changes
- LEE → Project Bridge → modify approved files
- LEE → Project Bridge → restart a service
- LEE → Project Bridge → deploy where authorized

The rule should be obvious:

Using CIL: LEE → CIL API

Repairing CIL: LEE → Project Bridge → CIL project

Using CerbaSeal: LEE → CerbaSeal API

Repairing CerbaSeal: LEE → Project Bridge → CerbaSeal project

Do not use the MCP management plane as a substitute for normal runtime APIs.

PHASE 3 — HARDEN THE CANONICAL DATABASE AND BRAIN LIFECYCLE

LEE's PostgreSQL-backed Brain must become essentially invisible to the owner during normal operation.

The owner should not normally configure, start, connect, restart, migrate, or maintain PostgreSQL manually.

The canonical application path must remain:

LEE Console / Android / Desktop
→ LEE API Server
→ canonical PostgreSQL database

The Console, Android application, and other clients must never connect directly to PostgreSQL.

Automatic desktop startup

When the owner opens the desktop application, the runtime supervisor must automatically:

- locate the canonical private LEE database configuration
- determine whether the expected private PostgreSQL instance is already running
- reuse it if it is healthy
- start the approved private PostgreSQL runtime if it is not running
- wait for PostgreSQL readiness
- verify that the database instance is actually LEE's expected database
- verify the expected database exists
- verify schema/database version
- determine whether migrations are required
- run approved migrations
- verify migration completion
- verify Event Log append-only protections

- verify critical Brain tables and records
- verify Brain/version metadata
- establish the API server database connection
- start the LEE API server
- load canonical Brain state
- start the Console
- report final database and Brain readiness

The intended owner experience is:

Double-click LEE → LEE starts herself.

No database button should need to be pressed.

Packaged migrations

Remove any remaining dependency on development-workspace migration behavior.

A packaged LEE installer must not require:

- pnpm
- npm commands
- developer tooling
- the source repository
- Replit access
- manual migration commands

Build a release-bundled migration runner.

Every migration should be:

- versioned
- deterministic
- auditable
- testable
- failure-aware
- compatible with backup and recovery

Before a migration with meaningful risk, LEE should create or verify a valid recovery point.

Database identity

LEE must prove she connected to her canonical database, not merely that a PostgreSQL server answered.

Use a stable installation/database identity or equivalent canonical marker.

This should prevent accidental connection to:

- the wrong PostgreSQL instance
- another application's database
- an empty accidental database
- an unexpected stale restore
- a different LEE installation

Strict database rules

LEE must never:

- silently create a substitute database because the canonical database failed
- silently create an empty replacement Brain
- reset the database simply to make startup pass

- discard existing historical data during migration without an explicit migration
- run two competing private PostgreSQL instances for the same LEE installation
- report Brain Ready before verification succeeds
- hide migration failure
- hide Event Log integrity failure
- silently continue normal writes when canonical persistence is unavailable
- fake healthy status because the UI is still able to load

If the canonical database cannot be safely loaded, LEE should enter a clear degraded/recovery state.

For example:

LEE Core — Degraded
Canonical Brain database unavailable.
No replacement Brain was created.

Database crash behavior

If PostgreSQL stops unexpectedly:

- detect the failure
- block unsafe database-dependent writes
- mark affected capabilities degraded
- attempt a bounded controlled restart
- verify canonical database identity again
- verify Event Log continuity
- reconnect the API
- restore normal status only after verification

If recovery fails, remain degraded and surface recovery actions.

Do not repeatedly restart forever.

Clean shutdown

When LEE exits:

- stop accepting new work requiring persistence
- finish or safely terminate active transactions
- flush required state
- stop the API server cleanly
- stop the private PostgreSQL runtime gracefully where LEE owns it
- preserve diagnostic information if shutdown failed

Window-close-to-tray behavior should not shut the database down accidentally.

PHASE 4 — MAKE BACKUP AND RECOVERY A FIRST-CLASS BRAIN CAPABILITY

The more useful LEE becomes, the more valuable her accumulated Brain becomes.

Backups cannot be treated as a secondary maintenance feature.

LEE should automatically protect:

- canonical database
- Event Log

- Brain versions
- critical configuration
- system manifests where needed
- provider configuration metadata excluding secrets
- knowledge structures
- projects
- people
- decisions
- commitments
- institutional knowledge

Build clear backup classes such as:

- routine automatic backup
- pre-migration recovery point
- pre-upgrade recovery point
- owner-created snapshot
- known-good recovery point

A backup should contain enough metadata to identify:

- creation time
- LEE version
- database/schema version
- Brain version
- Event Log checkpoint
- integrity/checksum information
- reason for backup
- restore compatibility

Do not call a checksum verification a restore test.

LEE should perform genuine restore validation in an isolated safe location where feasible.

The test should prove that the backup can actually reconstruct an operational database and expected Brain structures.

Restore behavior

Restore must be deliberate.

LEE must not automatically overwrite the canonical Brain because a backup exists.

The owner should be shown:

- backup date
- Brain/database version
- compatibility
- reason it was created
- integrity state
- what current data would be replaced

Restore should itself be auditable.

After restore:

- verify database identity
- verify schema
- verify Event Log
- verify Brain metadata
- verify critical tables

- run health/self-tests
- clearly report success or failure

Machine-loss recovery

Design toward the eventual experience:

New K6 or replacement computer
→ install LEE
→ choose Restore Existing LEE
→ select trusted backup
→ restore Brain
→ reconnect/re-authorize external accounts where secrets cannot safely travel
→ verify systems
→ LEE continues with historical knowledge intact

The Brain should outlive the hardware.

PHASE 5 — BUILD A CROSS-SYSTEM CORRELATION / RELATIONSHIP GRAPH

LEE can already ingest Gmail, Calendar, Drive, GitHub, Replit projects, people, project data, evidence, and other information.

Now make LEE understand when records from different systems refer to the same real-world activity.

Build or strengthen a provenance-backed Context Relationship Graph.

It should connect entities such as:

- people
- organizations
- projects
- initiatives
- emails
- email threads
- meetings
- documents
- repositories
- commits
- issues
- pull requests
- decisions
- assumptions
- evidence
- commitments
- waiting loops
- risks
- systems
- actions
- deployments

Relationships can include concepts such as:

- belongs to
- related to
- discussed in

- promised in
- caused by
- created from
- depends on
- blocked by
- waiting on
- supports
- contradicts
- supersedes
- owned by
- affects

Every durable relationship must have provenance.

LEE must distinguish:

- confirmed relationships
- strong inferred relationships
- weak candidate relationships
- owner-declared relationships

Do not silently promote uncertain relationship guesses into canonical truth.

This should make cross-system reconstruction possible.

Example:

Olivia email
→ Olivia person record
→ Line Axia organization
→ CerbaSeal project
→ pilot initiative
→ pilot-readiness document
→ related decisions
→ current waiting loop

PHASE 6 — MAKE “WHAT CHANGED?” A FIRST-CLASS CAPABILITY

Persistent state should let LEE answer:

- What changed today?
- What changed since yesterday?
- What changed since I last opened LEE?
- What changed in QuantraCore?
- What changed with Olivia?
- What changed with CerbaSeal?
- What changed across Lamont Labs this week?

Build persistent state-diff/change intelligence rather than rebuilding the answer from scratch every time.

Changes should have:

- previous state
- current state
- source
- timestamp

- affected entity
- significance
- confidence
- causal relationship where known

PHASE 7 — ADD CHANGE SIGNIFICANCE

Not every connector event deserves attention.

LEE needs a significance model.

Classify meaningful changes along a small scale such as:

Routine
Notable
Important
Critical

Examples:

Routine: minor repository metadata update

Notable: meaningful commit in an active project

Important: pilot contact replies with a substantive change

Critical: canonical database corruption, CerbaSeal failure during a required governed action, severe project failure, security-impacting state

Significance must be evidence-based and explainable.

Do not allow every GitHub event or email to compete equally for attention.

PHASE 8 — TURN TODAY INTO THE ACTUAL COMMAND CENTER

Today should answer:

What deserves Jesse's attention right now?

Do not turn Today into another metrics dashboard.

Use four primary sections.

NEEDS YOU

Only items requiring genuine owner involvement.

Examples:

- approval required
- important decision required
- CerbaSeal HOLD awaiting owner action
- email draft ready for approval
- project blocked by owner decision
- connector needs reauthorization
- unresolved important contradiction
- consequential action awaiting approval

- significant relationship requiring response

Keep this section intentionally small.

IN MOTION

Meaningful work that is actively progressing.

Examples:

- active project work
- current project tasks
- recent meaningful progress
- current build activity
- important upcoming meetings
- pilot activity
- major deliverables
- LEE-managed work currently underway

Do not show every raw event.

WATCHING

Things that do not require action yet but deserve awareness.

Examples:

- waiting loops
- important unanswered correspondence
- project inactivity
- dependency risk
- approaching deadline
- degraded service
- stale assumption
- build instability
- unusual model cost
- meeting needing preparation

LEE SUMMARY

Generate a concise operational brief based on evidence.

Example:

“Four meaningful things changed since yesterday. QuantraCore’s CI recovered. The CerbaSeal pilot remains in a waiting state and no follow-up is recommended yet. One Gmail connection requires reauthorization. Your next important commitment is Thursday.”

Today should become the primary place the owner starts the day.

PHASE 9 — MAKE ASK LEE THE UNIVERSAL INTERFACE

Ask LEE should become the easiest way to retrieve information across the environment.

LEE should be able to answer questions such as:

- What is happening with Olivia?
- Where does the CerbaSeal pilot stand?

- What am I waiting on?
- What do I owe people?
- What changed in QuantraCore?
- Which projects are declining?
- Which projects need my decision?
- Why is this project classified as stalled?
- What evidence supports this conclusion?
- What assumptions are affecting this recommendation?
- What did Replit change?
- Which connections are unhealthy?
- How much frontier-model usage did CIL avoid?
- What should I focus on today?
- What has changed since I last checked?

Ask LEE should retrieve across domains through the Query Engine rather than implementing unrelated chat logic for every subsystem.

Where useful, answers should expose:

- evidence
- freshness
- assumptions
- contradictions
- confidence
- relevant project/person
- explanation of why the conclusion was reached

The default answer should remain clean.

Detailed evidence should be expandable rather than dumped into every response.

PHASE 10 — BUILD COMMITMENT AND WAITING INTELLIGENCE

Use Gmail, Calendar, Projects, People, Decisions, and other evidence to create a proper commitment model.

LEE should distinguish:

- Jesse owes someone something
- Someone owes Jesse something
- Both parties are waiting on an external condition
- A task exists but no interpersonal commitment exists
- LEE inferred a possible commitment but confidence is not high enough to treat it as fact

A commitment should include:

- actor
- recipient
- commitment
- source evidence
- created date
- expected date if stated or safely inferred
- linked person
- linked organization
- linked project
- current status
- confidence

- last meaningful activity
- completion evidence

LEE should understand language such as:

“I’ll get that to you.”

“I’ll follow up next week.”

“I’ll review this.”

“Put that together and send it over.”

Do not create nagging noise from every conversational statement.

Waiting should consider:

- importance
- elapsed time
- relationship cadence
- expected response time
- project impact
- deadline proximity
- prior communication pattern

PHASE 11 — MAKE PEOPLE A REAL RELATIONSHIP-INTELLIGENCE SYSTEM

People should become much more than contact records.

For each important person, LEE should be able to reconstruct:

- identity
- organization
- role
- related projects
- related initiatives
- interaction history
- important messages
- relevant documents
- meetings
- commitments
- unresolved questions
- waiting loops
- decisions involving them
- recent communication
- upcoming interactions
- relationship cadence
- relevant evidence

A request such as:

“Where do things stand with Olivia?”

should reconstruct the relationship from evidence rather than rely on isolated stored summaries.

PHASE 12 — BUILD PROJECT OPERATIONAL STATE

LEE should understand project health from actual evidence.

Use signals including:

- current tasks
- task completion
- recent commits
- meaningful code changes
- build state
- CI state
- test state
- deployment state
- dependencies
- blockers
- waiting loops
- owner decisions required
- external dependencies
- recent progress
- inactivity
- errors
- unresolved failures

Support explainable momentum states such as:

- Explosive
- Rising
- Stable
- Declining
- Dormant
- Stalled

Every classification must be explainable.

Example:

QuantraCore — Rising

Webull architecture planning completed.
CI is passing.
Several implementation tasks are ready.
No unresolved build blocker exists.
Meaningful activity occurred during the last three days.

Do not create arbitrary AI-generated project-status labels without evidence.

PHASE 13 — TURN THE PROJECT BRIDGE INTO A PROJECT OPERATOR

Define one consistent project-operations contract.

Depending on project permission, LEE should eventually be able to:

- inspect project
- read manifest

- summarize current state
- search project
- read files
- inspect dependencies
- inspect logs
- inspect CI
- inspect build status
- run tests
- run typecheck
- run approved diagnostics
- preview code changes
- apply approved changes
- restart a service
- inspect deployment
- deploy where explicitly authorized
- compare system/API contracts
- verify an integration between projects

Use capability levels:

OBSERVE
Read and inspect.

USE
Run safe tools and checks.

MANAGE
Perform approved project modifications.

GOVERNED_MANAGE
Perform higher-risk actions such as production deployment or other consequential operations.

Project registration must remain explicit.

LEE must not automatically gain modification authority merely because she can read a repository.

PHASE 14 — BUILD THE PROJECT REPAIR LOOP

Create a governed repair workflow.

Target sequence:

Failure observed
→ inspect affected project
→ gather evidence
→ diagnose likely cause
→ determine whether repair is inside LEE's authority
→ prepare proposed change
→ preview diff
→ run safe checks
→ request approval when required
→ apply change
→ rerun tests/checks
→ verify project health
→ record Event Log/audit evidence
→ report result

LEE should eventually be able to say:

“QuantraCore CI failed because of a dependency mismatch. I prepared a two-file repair. All checks now pass. Review the change?”

Do not jump immediately to broad autonomous repair authority.

Build the deterministic repair machinery first.

PHASE 15 — BUILD ONE UNIFIED APPROVAL INBOX

Consequential actions from different parts of LEE should converge into one owner-facing approval queue.

Examples:

- Email send approval
- Project code change approval
- Deployment approval
- External provider mutation
- Important deletion
- Governed account action

Each approval should clearly show:

- requested action
- target
- reason
- affected system
- risk level
- proposed change
- evidence
- CerbaSeal status where required
- expiration
- whether owner confirmation is required
- what will happen after approval

Approvals should be available on desktop and Android.

Do not force the owner to find approvals inside individual subsystem pages.

PHASE 16 — SIMPLIFY THE CONNECTION CENTER

The underlying connection system can remain sophisticated.

The owner-facing experience should be simple.

Normal Connection Center should look approximately like:

Gmail — Connected

Google Drive — Connected / Read-only

Google Calendar — Connected / Read-only

GitHub — Connected

Replit Projects — Connected

CIL — Healthy

CerbaSeal — Healthy

Clicking a connection should show:

- status
- capabilities
- permissions
- last sync
- last successful operation
- last error
- reauthorize/reconnect
- disconnect where appropriate

Move technical information such as raw scopes, endpoints, manifests, contract details, correlation IDs, protocol diagnostics, and transport data into Advanced.

Connections should explicitly expose authority such as:

- OBSERVE
- USE
- MANAGE
- GOVERNED_MANAGE

Connection health must not imply mutation authority.

PHASE 17 — IMPLEMENT LAYERED READINESS

Do not reduce LEE to one simplistic Ready/Not Ready flag.

Create layered readiness.

CORE

Includes:

- canonical database
- Event Log
- Brain
- API server
- Console

INTELLIGENCE

Includes:

- CIL
- valid model route execution capability
- Replit AI Bridge or other approved execution route

GOVERNANCE

Includes:

- CerbaSeal
- required governance availability

CONNECTIONS

Includes:

- Gmail
- Calendar
- Drive
- GitHub
- other enabled providers

PROJECT OPERATIONS

Includes:

- Replit/MCP project bridge
- registered project capability health

DESKTOP

Includes:

- runtime supervisor
- private database
- migration state
- updater
- local services

Then LEE can report:

LEE READY

or:

LEE DEGRADED — Google Calendar requires reauthorization

while still showing that Core, Intelligence, Governance, and Project Operations are healthy.

A failure in an optional connection must not falsely imply the canonical Brain is down.

A failure in the canonical Brain must not be hidden behind healthy optional connections.

PHASE 18 — BUILD OPERATIONAL CONFIDENCE

Create an Operational Confidence measurement representing how complete and current LEE's understanding of reality is.

This is different from confidence in an individual answer.

Possible inputs include:

- connector freshness
- Gmail synchronization
- Calendar synchronization
- Drive synchronization
- GitHub/project visibility
- Event Log continuity
- database health
- stale facts
- unresolved contradictions

- unavailable systems
- missing project state
- unknown waiting-loop state
- stale assumptions

Example:

Operational Confidence — 93/100

“Gmail, Calendar, GitHub, CIL, CerbaSeal, and the project bridge are current. Google Drive has not synchronized in 18 hours and two project assumptions are stale.”

The score must be explainable and evidence-based.

Do not manufacture precision unsupported by the underlying inputs.

PHASE 19 — MAKE EVIDENCE POWERFUL BUT QUIET

Keep LEE’s provenance/evidence architecture.

Do not force the owner to constantly stare at evidence IDs and technical metadata.

Normal experience:

“CerbaSeal’s pilot opportunity appears active but slow-moving.”

Then the owner can choose:

Why?

LEE can show the relevant email, document, decision, timeline events, or other evidence.

The evidence system should remain extremely strong underneath while the normal UI remains simple.

PHASE 20 — BUILD A MEANINGFUL UNIFIED TIMELINE

Timeline should combine meaningful activity across:

- Gmail
- Calendar
- Drive
- GitHub
- Replit
- projects
- people
- decisions
- evidence
- documents
- governance
- LEE’s own actions

Do not dump every connector event into the timeline.

Use significance filtering and entity context.

Allow project-specific, person-specific, organization-specific, and lab-wide timelines.

PHASE 21 — KEEP ANDROID FOCUSED

Do not duplicate the desktop Console on Android.

Primary mobile areas should remain approximately:

- Today
- Ask LEE
- Capture
- Approvals
- Alerts
- Systems

Capture should support:

- text
- voice note
- screenshot
- photo
- URL
- file
- idea
- observation
- quick project update

Offline capture should remain important.

The mobile companion should be optimized for how the owner actually uses a phone: quick input, awareness, questions, and approvals.

PHASE 22 — FINISH THE DESKTOP PRODUCT EXPERIENCE

The final target experience should be:

Download installer
→ install
→ desktop shortcut appears
→ open LEE
→ private PostgreSQL starts automatically
→ canonical database verified
→ migrations run automatically if required
→ Event Log verified
→ Brain loads
→ API server starts
→ Console starts
→ CIL checked
→ CerbaSeal checked
→ Replit/project bridge checked
→ providers checked
→ Today opens

The owner should not need to understand or manually operate:

- PostgreSQL
- Node

- npm
- pnpm
- ports
- migration commands
- PowerShell
- local server commands
- source repositories
- runtime process management

Windows should remain the strongest first turnkey target if that is where bundled PostgreSQL support is already most complete.

Expand macOS/Linux self-contained packaging only when it does not destabilize Windows/K6 completion.

PHASE 23 — DESKTOP RECOVERY AND SELF-REPAIR

Create a safe recovery mode for situations where normal startup cannot complete.

Recovery mode should be capable of inspecting:

- database availability
- database identity
- schema/migration state
- Event Log integrity
- Brain metadata
- backup availability
- API startup failure
- local runtime logs
- CIL state
- CerbaSeal state
- Replit Bridge state
- provider state

Recovery mode must favor diagnosis and preservation.

It must not automatically:

- reset the Brain
- delete the database
- discard Event Log history
- overwrite good backups
- remove configuration
- wipe credentials

The most destructive recovery actions should require explicit owner confirmation and clear explanation.

PHASE 24 — MAKE SYSTEM HEALTH ACTIONABLE

Healthy systems should be quiet.

Problems should be prominent.

Do not devote large amounts of Today or the main navigation to showing green checks.

If everything is healthy, LEE can simply report:

Systems healthy

If something changes:

CerbaSeal unavailable — governed external actions are currently blocked.

Gmail authorization expired — email synchronization paused.

Project Bridge degraded — LEE can inspect local knowledge but cannot currently work on Replit projects.

Every health warning should explain:

- what failed
- what remains available
- what is blocked
- whether LEE can recover automatically
- whether owner action is required

PHASE 25 — FINAL UX CONSOLIDATION

The primary desktop navigation should remain deliberately small.

Target approximately:

- Today
- Ask LEE
- Work
  - Projects
  - Portfolio
  - People
  - Objectives
- Knowledge
- Systems
- Settings

Move technical internals into Settings → Advanced or contextual diagnostics.

Avoid restoring a sidebar full of:

- raw Event Log
- Constitution
- Confidence
- assumptions
- decision impact
- policies
- evidence
- raw governance
- connectors
- costs
- backups
- scheduler internals
- knowledge map
- observations
- simulations
- reflections
- raw system economics
- identity internals

- engine registries
- capability registries

The functionality should remain.

The clutter should not.

THINGS NOT TO PRIORITIZE IN THIS PASS

Do not let these distract from operational completion:

- voice / ElevenLabs
- large numbers of new connectors
- additional standalone engines
- unnecessary AI modes
- exotic visualizations
- dashboard expansion for its own sake
- broad autonomous external writing
- duplicating every desktop capability on Android
- rebuilding functionality that already works
- cosmetic refactoring with no operational benefit

LEE already has enough organs.

This phase should build the nervous system connecting them.

FINAL TARGET EXPERIENCE

The finished direction should make interactions like this possible:

The owner opens LEE in the morning.

LEE says:

Three things need your attention.

Olivia has not replied since the latest CerbaSeal exchange. Based on the current communication history, no follow-up is recommended yet.

QuantraCore's current implementation plan is ready for approval. No project changes have been made.

Project LEE's CI recovered from the previous failure and the current build is healthy.

You have one important meeting this afternoon. I found two relevant emails, the current Drive document, and one unresolved commitment associated with it.

Then the owner asks:

Check my projects. Anything broken?

LEE answers:

QuantraCore, CerbaSeal, and CIL are healthy.

Greyline has one failed workflow. I inspected the project and identified a dependency mismatch. I prepared a repair and the safe checks pass. Review the proposed change?

That is the product target.

LEE should not merely store everything.

LEE should:

observe broadly
→ reconstruct reality
→ determine significance
→ understand relationships
→ identify what matters
→ prepare useful work
→ safely perform what she is authorized to perform
→ bring the owner only what genuinely requires the owner.

Before marking this implementation pass complete, run the full existing validation suite plus new integration, startup, database, backup/restore, CIL-routing, project-operation, approval, and failure-injection tests. Report what was completed, what remains partial, any deliberate deviations from this specification, and the exact evidence supporting completion.